

Aperture — CI/CD & DevOps Guide

Version: v1.0

Document Type: CI/CD & DevOps

Related Documents

- Backend Architecture
 - Frontend Architecture
 - Testing Strategy
 - Technical Architecture
-

1. Purpose

This document defines how Aperture is built, tested, deployed, monitored, and operated.

It establishes the engineering workflow from local development through production deployment while ensuring repeatability, reliability, and automation.

The objective is to make deployments routine rather than exceptional.

2. DevOps Philosophy

Aperture follows several operational principles:

- Infrastructure as code
- Automated testing
- Automated deployments
- Immutable deployments
- Reproducible environments
- Observability by default
- Small, incremental releases

Operational complexity should grow only when justified by product needs.

3. Environment Strategy

The application is deployed across multiple environments.

```text id="8yl8o7" Developer

↓

Local

↓

Preview

↓

Staging

↓

Production

Each environment serves a distinct purpose.

---

### Local

Used for development.

Runs entirely through Docker Compose.

---

### Preview

Automatically generated for pull requests.

Used for design review and feature validation.

---

### Staging

Production-like environment.

Used for release validation.

---

### ### Production

Public deployment.

Receives only validated releases.

---

### # 4. Local Development

Every contributor should be able to run the application with minimal setup.

Required steps:

- Clone repository
- Configure environment variables
- Run Docker Compose
- Execute database migrations
- Start development servers

Local environments should closely resemble production while remaining simple to use.

---

### # 5. Source Control Strategy

Git workflow:

```text id="y9m6ll"  
main

↓

develop

↓

feature/*

Rules:

- Small pull requests
- One logical change per branch
- Protected main branch
- Code review required

History should remain readable and meaningful.

6. Continuous Integration

Every push triggers an automated pipeline.

Pipeline stages:

Checkout

↓

Install Dependencies

↓

Lint

↓

Format Check

↓

Type Check

↓

Backend Tests

↓

Frontend Tests

↓

Integration Tests

↓

Build

↓

Docker Build

↓

Success

Only passing pipelines are eligible for merge.

7. Continuous Deployment

Deployment is automated.

Typical flow:

```text id="t0w5nc"

Merge to Main

↓

Build

↓

Container Image

↓

Registry

↓

Deploy

↓

Health Check

↓

Production

Rollback procedures should be documented and tested.

## 8. Container Strategy

Docker is the standard runtime environment.

Separate containers include:

- Frontend
- Backend
- PostgreSQL
- Redis

Production deployments should use versioned container images.

Containers are immutable once built.

---

## 9. Infrastructure as Code

Infrastructure is managed declaratively.

Examples:

- Compute resources
- Databases
- Networking
- Storage
- Monitoring

Infrastructure changes should undergo the same review process as application code.

---

## 10. Configuration Management

Configuration is externalized.

Examples:

- Database URLs
- API keys
- JWT secrets
- Redis endpoints
- Feature flags

Configuration should never be hardcoded into the application.

Secrets should be managed securely and rotated periodically.

---

## 11. Database Migrations

Schema changes are applied through version-controlled migrations.

Deployment process:

1. Backup
2. Apply migration
3. Validate
4. Deploy application

Destructive schema changes should be introduced gradually.

---

## 12. Release Strategy

Releases should be:

- Small
- Frequent
- Reversible

Every release should include:

- Release notes
- Migration review
- Health verification

Large infrequent releases increase operational risk.

---

## 13. Observability

Operational visibility relies on three pillars:

### Logs

Structured, centralized logs.

Include:

- Request ID
  - User ID (when available)
  - Duration
  - Status
- 

## Metrics

Track:

- Request latency
  - Error rates
  - Throughput
  - Cache hit ratio
  - Database performance
- 

## Tracing

Future support for distributed tracing as services are extracted.

---

# 14. Health Checks

Operational endpoints include:

- Liveness
- Readiness
- Version
- Dependency health

Health checks should validate critical dependencies without performing expensive operations.

---

# 15. Monitoring

Dashboards should provide visibility into:

- Application health
- Infrastructure health
- Background jobs
- Search
- Recommendation pipelines



- Database
- Cache

Alerts should prioritize actionable issues rather than generating excessive noise.

---

## 16. Security Operations

Operational security includes:

- Secret rotation
- Dependency scanning
- Container scanning
- TLS enforcement
- Access auditing
- Least privilege
- Protected administrative interfaces

Security reviews should accompany major architectural changes.

---

## 17. Disaster Recovery

Preparation includes:

- Automated backups
- Restore verification
- Rollback procedures
- Infrastructure recreation

Recovery procedures should be documented and periodically exercised.

---

## 18. Operational Runbooks

Standard operational procedures should exist for:

- Deployment failures
- Database migration failures
- External API outages
- High error rates
- Cache failures
- Search degradation

Runbooks reduce recovery time during incidents.

---

## 19. Cost Management

Infrastructure should scale efficiently.

Guidelines:

- Prefer managed services initially
- Avoid idle resources
- Monitor storage growth
- Review cloud costs regularly

Engineering decisions should consider both performance and operational cost.

---

## 20. Future Evolution

Potential additions:

- Blue/green deployments
- Canary releases
- Multi-region deployment
- Kubernetes
- Service mesh
- Automated scaling

These should be introduced only when operational complexity is justified.

---

## 21. Interview Discussion Points

Be prepared to explain:

- Why automate deployments?
  - Why keep infrastructure as code?
  - Why separate staging from production?
  - How would you safely deploy a database migration?
  - Why use immutable container images?
  - How would you roll back a failed release?
-

## 22. Summary

The Aperture CI/CD and DevOps strategy emphasizes automation, consistency, and operational reliability.

By combining containerization, automated testing, infrastructure as code, continuous deployment, and observability, the platform is designed to support frequent, low-risk releases while remaining approachable for an individual developer.

The operational practices described here are intentionally incremental, allowing the infrastructure to evolve alongside the application rather than outpacing it.